

Dummy title

Anonymous author

Anonymous affiliation

Anonymous author

Anonymous affiliation

Abstract

Статическая схема синхронизации заранее закрепляет части упорядоченной области данных за фиксированным набором блокировок. При неравномерном распределении запросов независимые обращения к различным часто используемым элементам могут конкурировать за одну блокировку, в то время как другие блокировки почти не задействованы. Кроме того, конфигурация, подходящая для одного этапа работы, может перестать соответствовать нагрузке после её изменения.

Мы предлагаем рассматривать соответствие между фиксированным набором блокировок и участками упорядоченной области как самостоятельный адаптивный слой синхронизации. Во время работы он изменяет это соответствие, опираясь на наблюдаемое распределение точечных и диапазонных запросов. Цель состоит в том, чтобы снизить конкуренцию между обращениями к различным данным, не допуская чрезмерных расходов на синхронизацию диапазонных операций. Число блокировок, сами защищаемые данные и их внутреннее представление при этом остаются неизменными.

Подход ориентирован на нагрузки, структура которых сохраняется на достаточно продолжительных этапах: недавние обращения позволяют судить о распределении ближайших запросов, хотя между этапами оно может изменяться или медленно смещаться. Примерами служат несколько горячих областей с меняющейся относительной интенсивностью и движущееся рабочее множество с преимущественно локальными диапазонными обращениями. Адаптация может повышать производительность, когда такая структура сохраняется достаточно долго, чтобы снижение конкуренции окупало её накладные расходы. В наших экспериментах при равномерной нагрузке адаптация не даёт преимущества, но её производительность остаётся близкой к статическому равномерному разбиению.

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements Anonymous acknowledgements

1 Обзор подхода

Синхронизация при фиксированном наборе блокировок.

Рассматривается конкурентная структура данных, элементы которой соответствуют позициям упорядоченной области $D = [0, N)$. Каждая операция заранее указывает диапазон $[l, r)$, к которому она будет обращаться. Структура блокировок не владеет защищаемыми данными и не меняет их расположение: ей известны только размер области и указанный операцией диапазон.

Для синхронизации используется фиксированный набор из K блокировок. Их конфигурация разбивает D на K последовательных частей, каждая из которых защищается своей блокировкой. Вначале части имеют приблизительно одинаковый размер. Во время работы их границы могут перемещаться, но число блокировок остаётся неизменным. Перед началом операции структура захватывает блокировки всех частей, пересекающих указанный диапазон, и освобождает их после завершения.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:10

Leibniz International Proceedings in Informatics



Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

От положения границ зависит доступный параллелизм. При равномерном разбиении несколько часто используемых элементов могут попасть под одну блокировку, тогда как блокировки редко используемых частей почти не задействованы. Перенос границ позволяет распределить такую нагрузку между несколькими блокировками. С другой стороны, после переноса широкий запрос может пересекать больше частей и захватывать больше блокировок.

52 Модель нагрузки.

Обозначим через $P_t(l, r)$ распределение диапазонов запросов в момент t . Оно заранее неизвестно и может меняться со временем. Мы рассматриваем нагрузку, состоящую из достаточно длинных этапов. При переходе к новому этапу распределение может измениться резко, а внутри этапа — только медленно. Форма P_t не задаётся, и соседние запросы не обязаны быть независимыми. Требуется лишь, чтобы по недавним запросам можно было судить о ближайших будущих запросах.

Не любая неравномерная нагрузка допускает полезную перестройку. Должно существовать разбиение D на K частей, которое распределяет обращения по блокировкам заметно равномернее, чем текущее разбиение, и при этом не заставляет часто встречающиеся диапазоны пересекать слишком много частей. Такой характер нагрузки должен сохраняться достаточно долго, чтобы выигрыш окупил сбор статистики и перестройку. Примеры — несколько разнесённых горячих областей, интенсивность которых меняется от этапа к этапу, и запросы к коротким диапазонам внутри медленно движущегося рабочего окна. Если подходящего разбиения нет или нагрузка меняется слишком быстро, выигрыша не ожидается.

68 Наблюдение за нагрузкой.

Для сбора статистики область D заранее делится на B фиксированных ячеек. Это деление не меняется при перестройке, а границы частей можно размещать только между соседними ячейками. Для каждой ячейки b хранится счётчик h_b . Попавший в выборку запрос увеличивает счётчики всех ячеек, которых он касается. Поэтому запрос к одному элементу увеличивает один счётчик, а более широкий запрос — счётчики всех ячеек между его концами.

Для каждой возможной границы b также хранится счётчик x_b . Он увеличивается, если запрос из выборки пересекает эту границу. Значения h_b показывают, к каким частям области обращаются чаще, а значения x_b — как часто запросу потребуется дополнительная блокировка при выборе данной границы.

Чтобы снизить накладные расходы, можно учитывать только заданную долю запросов. Счётчики нужны лишь для выбора нового разбиения и не влияют на корректность операций, поэтому согласованный мгновенный снимок всех значений не требуется. Они описывают диапазоны запросов, но не измеряют время ожидания блокировок. Поскольку ячейки остаются на месте, собранную статистику можно использовать для сравнения разных конфигураций.

85 Выбор разбиения.

86 По собранной статистике выбирается новое разбиение B ячеек на K последовательных
87 частей. Пусть $0 = c_0 < c_1 < \dots < c_K = B$ — их границы, а

$$88 \quad H_i = \sum_{b=c_{i-1}}^{c_i-1} h_b$$

89 — оценка нагрузки на блокировку i , полученная как сумма счётчиков её ячеек. Стоимость
90 разбиения равна

$$91 \quad \mathcal{C}(c_1, \dots, c_{K-1}) = \sum_{i=1}^K H_i^2 + \lambda \sum_{i=1}^{K-1} x_{c_i}. \quad (1)$$

92 Первая сумма штрафует неравномерное распределение нагрузки: большое значение
93 H_i быстро увеличивает стоимость. Для запросов к одному элементу её нормированное
94 значение равно вероятности того, что два независимых запроса потребуют одну блоки-
95 ровку. Для запросов к нескольким элементам это лишь приближённая оценка. Вторая
96 сумма штрафует границы, которые часто пересекают запросы: каждая такая граница
97 заставляет запрос захватить ещё одну блокировку. Параметр λ задаёт соотношение
98 между этими двумя требованиями.

99 Границы выбираются методом динамического программирования. Он находит раз-
100 биение минимальной стоимости среди всех разбиений на ровно K непустых частей.
101 Все границы выбираются совместно и при одной перестройке могут измениться од-
102 новременно. Время работы алгоритма составляет $O(KB^2)$, а объём дополнительной
103 памяти — $O(KB)$. В экспериментах мы использовали $B = 1024$ и $K = 64$ как рабочую
104 конфигурацию. Метод не привязан к этим значениям; они лишь показывают порядок
105 параметров, при котором разбиение остаётся достаточно подробным, а перестройка —
106 практически выполнимой.

107 Перестройка во время работы.

108 Фоновый поток периодически проверяет накопленную статистику. В автоматическом
109 режиме новое разбиение рассматривается только тогда, когда статистики достаточно,
110 а нагрузка заметно неравномерна. После первой подходящей выборки также прове-
111 ряется, изменилось ли распределение по сравнению с ранее сохранённой статистикой.
112 Вычисление происходит в фоне, пока запросы продолжают использовать текущую
113 конфигурацию. Найденное разбиение устанавливается только в том случае, если оно
114 достаточно улучшает целевую функцию. Эти проверки позволяют избежать частых
115 перестроек без ожидаемого выигрыша.

116 На время установки новые запросы приостанавливаются. Фоновый поток захватывает
117 все K блокировок и дожидается завершения запросов, уже работающих со старым
118 разбиением. После этого он заменяет границы и отображение ячеек на блокировки.
119 Защищаемые данные при этом не перемещаются.

120 **2** Связанные работы

121 Деревья поиска с адаптацией к конкуренции.

122 Прямая ссылка на PDF статьи.

Наиболее близким подходом среди параллельных структур данных является дерево, адаптирующееся к конкуренции, или СА-дерево [6]. Его маршрутизирующие узлы направляют операцию к базовому узлу, который содержит последовательное упорядоченное множество и защищается отдельно. Ожидание при захвате блокировки увеличивает локальную оценку конкуренции, тогда как захват без ожидания уменьшает её. При устойчиво высокой конкуренции содержимое базового узла делится между двумя новыми базовыми узлами с независимыми блокировками, а при устойчиво низкой содержимое соседних узлов объединяется в одном узле. Таким образом, посредством локальных преобразований изменяются и число областей синхронизации, и дерево, в котором хранятся данные. Позднее тот же принцип был перенесён в неблокирующее LFCA-дерево: базовые узлы стали ссылаться на неизменяемые контейнеры, а обновления устанавливают новые узлы с помощью операции сравнения с обменом [7]. Локальные разделения и объединения при этом сохранились, но поиск получил гарантию wait-free, а обновления и диапазонные запросы — lock-free.

Блокировка логических диапазонов.

Прямая ссылка на PDF статьи.

Не связанный с деревьями подход предложен Lomet и Mokbel в протоколе блокировки разделов [5]. Между уровнями таблицы и отдельной записи вводятся логические ресурсы, соответствующие непрерывным диапазонам ключей. Диапазонная операция может захватывать целиком внутренние разделы своего интервала, а его края обрабатывать более точными блокировками диапазонов ключей. Авторы также обсуждают динамическое разделение и объединение ресурсов: их размеры можно выбирать по числу записей или по гистограмме плотности ключей, чтобы компенсировать неравномерность хранимых данных. Такая перестройка требует согласовать блокировки текущих владельцев и очереди ожидающих транзакций.

Отличия предлагаемого подхода.

СА-деревья меняют гранулярность посредством локальных преобразований внутри конкретной структуры поиска: изменяются число базовых узлов, топология индекса и представление хранимых данных. Протокол Lomet и Mokbel отделяет логические диапазоны от физического хранения, но предлагаемое в нём размещение границ следует за распределением записей, а не за частотой обращений к ним. Наш подход сохраняет фиксированное число блокировок, не перемещает данные и совместно пересчитывает все границы по накопленной статистике запросов. Поэтому перестройка представляет собой глобальную задачу разбиения с фиксированным числом областей, а не последовательность локальных решений.

3 Typesetting instructions – Summary

LIPICs is a series of open access high-quality conference proceedings across all fields in informatics established in cooperation with Schloss Dagstuhl. In order to do justice to the high scientific quality of the conferences that publish their proceedings in the LIPICs series, which is ensured by the thorough review process of the respective events, we believe that LIPICs proceedings must have an attractive and consistent layout matching the standard of the series. Moreover, the quality of the metadata, the typesetting and the layout must also meet the requirements of other external parties such as indexing service, DOI registry,

166 funding agencies, among others. The guidelines contained in this document serve as the
 167 baseline for the authors, editors, and the publisher to create documents that meet as many
 168 different requirements as possible.

169 Please comply with the following instructions when preparing your article for a LIPIcs
 170 proceedings volume.

171 Minimum requirements

- 172 ■ Use pdf_latex and an up-to-date L^AT_EX system.
- 173 ■ Use further L^AT_EX packages and custom made macros carefully and only if required.
- 174 ■ Use the provided sectioning macros: `\section`, `\subsection`, `\subsubsection`,
 175 `\paragraph`, `\paragraph*`, and `\subparagraph*`.
- 176 ■ Provide suitable graphics of at least 300dpi (preferably in PDF format).
- 177 ■ Use BibTeX and keep the standard style (`plainurl`) for the bibliography.
- 178 ■ Please try to keep the warnings log as small as possible. Avoid overfull `\hboxes` and any
 179 kind of warnings/errors with the referenced BibTeX entries.
- 180 ■ Use a spellchecker to correct typos.

181 Mandatory metadata macros

182 Please set the values of the metadata macros carefully since the information parsed from
 183 these macros will be passed to publication servers, catalogues and search engines. Avoid
 184 placing macros inside the metadata macros. The following metadata macros/environments
 185 are mandatory:

- 186 ■ `\title` and, in case of long titles, `\titlerunning`.
- 187 ■ `\author`, one for each author, even if two or more authors have the same affiliation.
- 188 ■ `\authorrunning` and `\Copyright` (concatenated author names)
 189 The `\author` macros and the `\Copyright` macro should contain full author names (espe-
 190 cially with regard to the first name), while `\authorrunning` should contain abbreviated
 191 first names.
- 192 ■ `\ccsdesc` (ACM classification, see <https://www.acm.org/publications/class-2012>).
- 193 ■ `\keywords` (a comma-separated list of keywords).
- 194 ■ `\relatedversion` (if there is a related version, typically the “full version”); please make
 195 sure to provide a persistent URL, e.g., at arXiv.
- 196 ■ `\begin{abstract}... \end{abstract}`.

197 Please do not ...

198 Generally speaking, please do not override the `lipics-v2021`-style defaults. To be more
 199 specific, a short checklist also used by Dagstuhl Publishing during the final typesetting is
 200 given below. In case of **non-compliance** with these rules Dagstuhl Publishing will remove
 201 the corresponding parts of L^AT_EX code and **replace it with the lipics-v2021 defaults**.
 202 In serious cases, we may reject the LaTeX-source and expect the corresponding author to
 203 revise the relevant parts.

- 204 ■ Do not use a different main font. (For example, the `times` package is forbidden.)
- 205 ■ Do not alter the spacing of the `lipics-v2021.cls` style file.
- 206 ■ Do not use `enumitem` and `paralist`. (The `enumerate` package is preloaded, so you can
 207 use `\begin{enumerate}[(a)]` or the like.)
- 208 ■ Do not use “self-made” sectioning commands (e.g., `\noindent{\bf My Paragraph}`).
- 209 ■ Do not hide large text blocks using comments or `\iffalse ... \fi` constructions.

- 210 ■ Do not use conditional structures to include/exclude content. Instead, please provide
- 211 only the content that should be published – in one file – and nothing else.
- 212 ■ Do not wrap figures and tables with text. In particular, the package `wrapfig` is not
- 213 supported.
- 214 ■ Do not change the bibliography style. In particular, do not use author-year citations.
- 215 (The `natbib` package is not supported.)

216 This is only a summary containing the most relevant details. Please read the complete
 217 document “LIPICs: Instructions for Authors and the `lipics-v2021` Class” for all details and
 218 don’t hesitate to contact Dagstuhl Publishing (<mailto:publishing@dagstuhl.de>) in case
 219 of questions or comments: [http://drops.dagstuhl.de/styles/lipics-v2021/](http://drops.dagstuhl.de/styles/lipics-v2021/lipics-v2021-authors/lipics-v2021-authors-guidelines.pdf)
 220 [lipics-v2021-authors/lipics-v2021-authors-guidelines.pdf](http://drops.dagstuhl.de/styles/lipics-v2021/lipics-v2021-authors/lipics-v2021-authors-guidelines.pdf)

221 4 Lorem ipsum dolor sit amet

222 Lorem ipsum dolor sit amet, consectetur adipiscing elit [4]. Praesent convallis orci arcu, eu
 223 mollis dolor. Aliquam eleifend suscipit lacinia. Maecenas quam mi, porta ut lacinia sed,
 224 convallis ac dui. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse potenti.
 225 Donec eget odio et magna ullamcorper vehicula ut vitae libero. Maecenas lectus nulla, auctor
 226 nec varius ac, ultricies et turpis. Pellentesque id ante erat. In hac habitasse platea dictumst.
 227 Curabitur a scelerisque odio. Pellentesque elit risus, posuere quis elementum at, pellentesque
 228 ut diam. Quisque aliquam libero id mi imperdiet quis convallis turpis eleifend.

229 ► **Lemma 1** (Lorem ipsum). *Vestibulum sodales dolor et dui cursus iaculis. Nullam ullamcor-*
 230 *per purus vel turpis lobortis eu tempus lorem semper. Proin facilisis gravida rutrum. Etiam*
 231 *sed sollicitudin lorem. Proin pellentesque risus at elit hendrerit pharetra. Integer at turpis*
 232 *varius libero rhoncus fermentum vitae vitae metus.*

233 **Proof.** Cras purus lorem, pulvinar et fermentum sagittis, suscipit quis magna.

234 **Just some paragraph within the proof.** Nam liber tempor cum soluta nobis eleifend option
 235 congue nihil imperdiet doming id quod mazim placerat facer possim assum. Lorem ipsum
 236 dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut
 237 laoreet dolore magna aliquam erat volutpat.

238 ▷ **Claim 2.** content...

239 Proof. content...

240 1. abc abc abc

◁

241

◀

242 ► **Corollary 3** (Curabitur pulvinar, [2]). *Nam liber tempor cum soluta nobis eleifend option*
 243 *congue nihil imperdiet doming id quod mazim placerat facer possim assum. Lorem ipsum*
 244 *dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut*
 245 *laoreet dolore magna aliquam erat volutpat.*

246 ► **Proposition 4.** *This is a proposition*

247 Proposition 4 and Proposition 4 ...

■ **Listing 1** Useless code.

```
for i:=maxint to 0 do
begin
    j:=square(root(i));
end;
```

248 4.1 Curabitur dictum felis id sapien

249 Curabitur dictum Corollary 3 felis id sapien Corollary 3 mollis ut venenatis tortor feugiat.
 250 Curabitur sed velit diam. Integer aliquam, nunc ac egestas lacinia, nibh est vehicula nibh, ac
 251 auctor velit tellus non arcu. Vestibulum lacinia ipsum vitae nisi ultrices eget gravida turpis
 252 laoreet. Duis rutrum dapibus ornare. Nulla vehicula vulputate iaculis. Proin a consequat
 253 neque. Donec ut rutrum urna. Morbi scelerisque turpis sed elit sagittis eu scelerisque quam
 254 condimentum. Pellentesque habitant morbi tristique senectus et netus et malesuada fames
 255 ac turpis egestas. Aenean nec faucibus leo. Cras ut nisl odio, non tincidunt lorem. Integer
 256 purus ligula, venenatis et convallis lacinia, scelerisque at erat. Fusce risus libero, convallis at
 257 fermentum in, dignissim sed sem. Ut dapibus orci vitae nisl viverra nec adipiscing tortor
 258 condimentum [1]. Donec non suscipit lorem. Nam sit amet enim vitae nisl accumsan pretium.

259 4.2 Proin ac fermentum augue

260 Proin ac fermentum augue. Nullam bibendum enim sollicitudin tellus egestas lacinia euismod
 261 orci mollis. Nulla facilisi. Vivamus volutpat venenatis sapien, vitae feugiat arcu fringilla ac.
 262 Mauris sapien tortor, sagittis eget auctor at, vulputate pharetra magna. Sed congue, dui
 263 nec vulputate convallis, sem nunc adipiscing dui, vel venenatis mauris sem in dui. Praesent
 264 a pretium quam. Mauris non mauris sit amet eros rutrum aliquam id ut sapien. Nulla
 265 aliquet fringilla sagittis. Pellentesque eu metus posuere nunc tincidunt dignissim in tempor
 266 dolor. Nulla cursus aliquet enim. Cras sapien risus, accumsan eu cursus ut, commodo vel
 267 velit. Praesent aliquet consectetur ligula, vitae iaculis ligula interdum vel. Integer faucibus
 268 faucibus felis.

- 269 ■ Ut vitae diam augue.
- 270 ■ Integer lacus ante, pellentesque sed sollicitudin et, pulvinar adipiscing sem.
- 271 ■ Maecenas facilisis, leo quis tincidunt egestas, magna ipsum condimentum orci, vitae
 272 facilisis nibh turpis et elit.

273 ► Remark 5. content...

274 5 Pellentesque quis tortor

275 Nec urna malesuada sollicitudin. Nulla facilisi. Vivamus aliquam tempus ligula eget ornare.
 276 Praesent eget magna ut turpis mattis cursus. Aliquam vel condimentum orci. Nunc congue,
 277 libero in gravida convallis [3], orci nibh sodales quam, id egestas felis mi nec nisi. Suspendisse
 278 tincidunt, est ac vestibulum posuere, justo odio bibendum urna, rutrum bibendum dolor sem
 279 nec tellus.

280 ► **Lemma 6** (Quisque blandit tempus nunc). *Sed interdum nisl pretium non. Mauris sodales*
 281 *consequat risus vel consectetur. Aliquam erat volutpat. Nunc sed sapien ligula. Proin faucibus*
 282 *sapien luctus nisl feugiat convallis faucibus elit cursus. Nunc vestibulum nunc ac massa*

283 *pretium pharetra. Nulla facilis turpis id augue venenatis blandit. Cum sociis natoque*
 284 *penatibus et magnis dis parturient montes, nascetur ridiculus mus.*

285 Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui. Nam
 286 vulputate, velit et laoreet porttitor, quam arcu facilis dui, sed malesuada risus massa sit
 287 amet neque.

288 **6 Morbi eros magna**

289 Morbi eros magna, vestibulum non posuere non, porta eu quam. Maecenas vitae orci risus,
 290 eget imperdiet mauris. Donec massa mauris, pellentesque vel lobortis eu, molestie ac turpis.
 291 Sed condimentum convallis dolor, a dignissim est ultrices eu. Donec consectetur volutpat
 292 eros, et ornare dui ultricies id. Vivamus eu augue eget dolor euismod ultrices et sit amet nisi.
 293 Vivamus malesuada leo ac leo ullamcorper tempor. Donec justo mi, tempor vitae aliquet non,
 294 faucibus eu lacus. Donec dictum gravida neque, non porta turpis imperdiet eget. Curabitur
 295 quis euismod ligula.

296 **References**

- 297 **1** Edsger W. Dijkstra. Letters to the editor: go to statement considered harmful. *Commun.*
 298 *ACM*, 11(3):147–148, 1968. doi:10.1145/362929.362947.
- 299 **2** Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan
 300 Kaufmann, 1993.
- 301 **3** John E. Hopcroft, Wolfgang J. Paul, and Leslie G. Valiant. On time versus space and
 302 related problems. In *16th Annual Symposium on Foundations of Computer Science, Berkeley,*
 303 *California, USA, October 13-15, 1975*, pages 57–64. IEEE Computer Society, 1975. doi:
 304 10.1109/SFCS.1975.23.
- 305 **4** Donald E. Knuth. Computer Programming as an Art. *Commun. ACM*, 17(12):667–673, 1974.
 306 doi:10.1145/361604.361612.
- 307 **5** David Lomet and Mohamed F. Mokbel. Locking key ranges with unbundled transaction services.
 308 *Proceedings of the VLDB Endowment*, 2(1):265–276, 2009. doi:10.14778/1687627.1687658.
- 309 **6** Konstantinos Sagonas and Kjell Winblad. Contention adapting search trees. In *14th Interna-*
 310 *tional Symposium on Parallel and Distributed Computing*, pages 215–224. IEEE Computer
 311 Society, 2015. doi:10.1109/ISPDC.2015.32.
- 312 **7** Kjell Winblad, Konstantinos Sagonas, and Bengt Jonsson. Lock-free contention adapting search
 313 trees. In *Proceedings of the 30th Symposium on Parallelism in Algorithms and Architectures*,
 314 pages 121–132. ACM, 2018. doi:10.1145/3210377.3210413.

315 **A Styles of lists, enumerations, and descriptions**

316 List of different predefined enumeration styles:

317 **■** `\begin{itemize}...\end{itemize}`

318 **■** ...

319 **■** ...

320 **1.** `\begin{enumerate}...\end{enumerate}`

321 **2.** ...

322 **3.** ...

323 **(a)** `\begin{alphaenumerate}...\end{alphaenumerate}`

324 (b) ...

325 (c) ...

326 (i) \begin{romanenumerate}...\end{romanenumerate}

327 (ii) ...

328 (iii) ...

329 (1) \begin{bracketenumerate}...\end{bracketenumerate}

330 (2) ...

331 (3) ...

332 **Description 1** \begin{description} \item[Description 1] ... \end{description}

333 **Description 2** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.

334 Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus

335 massa sit amet neque.

336 **Description 3** ...

337 Proposition 10 and Proposition 10 ...

338 **B Theorem-like environments**

339 List of different predefined enumeration styles:

340 ► **Theorem 7.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.*

341 *Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa*

342 *sit amet neque.*

343 ► **Lemma 8.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.*

344 *Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa*

345 *sit amet neque.*

346 ► **Corollary 9.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.*

347 *Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa*

348 *sit amet neque.*

349 ► **Proposition 10.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo*

350 *dui. Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus*

351 *massa sit amet neque.*

352 ► **Conjecture 11.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo*

353 *dui. Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus*

354 *massa sit amet neque.*

355 ► **Observation 12.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et*

356 *leo dui. Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus*

357 *massa sit amet neque.*

358 ► **Exercise 13.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo*

359 *dui. Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus*

360 *massa sit amet neque.*

361 ► **Definition 14.** *Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo*

362 *dui. Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus*

363 *massa sit amet neque.*

364 ► **Example 15.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo
 365 dui. Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus
 366 massa sit amet neque.

367 ► **Note 16.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.
 368 Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa
 369 sit amet neque.

370 ► **Note.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui. Nam
 371 vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa sit
 372 amet neque.

373 ► **Remark 17.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.
 374 Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa
 375 sit amet neque.

376 ► **Remark.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.
 377 Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa
 378 sit amet neque.

379 ▷ **Claim 18.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.
 380 Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa
 381 sit amet neque.

382 ▷ **Claim.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui.
 383 Nam vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa
 384 sit amet neque.

385 **Proof.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui. Nam
 386 vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa sit
 387 amet neque. ◀

388 **Proof.** Fusce eu leo nisi. Cras eget orci neque, eleifend dapibus felis. Duis et leo dui. Nam
 389 vulputate, velit et laoreet porttitor, quam arcu facilisis dui, sed malesuada risus massa sit
 390 amet neque. ◀